Linux User Management

Types of users, creating and managing accounts, /etc/passwd and /etc/shadow, passwd and chage, home directories, command chart and interview Q&A

Privilege note: commands are written without sudo, as asked. That does not make them unprivileged.
ROOT = changes accounts or reads protected files; run from a root shell.
ANY USER = safe for everyone. Replace words in <angle brackets> with real names.

1. What is user management?

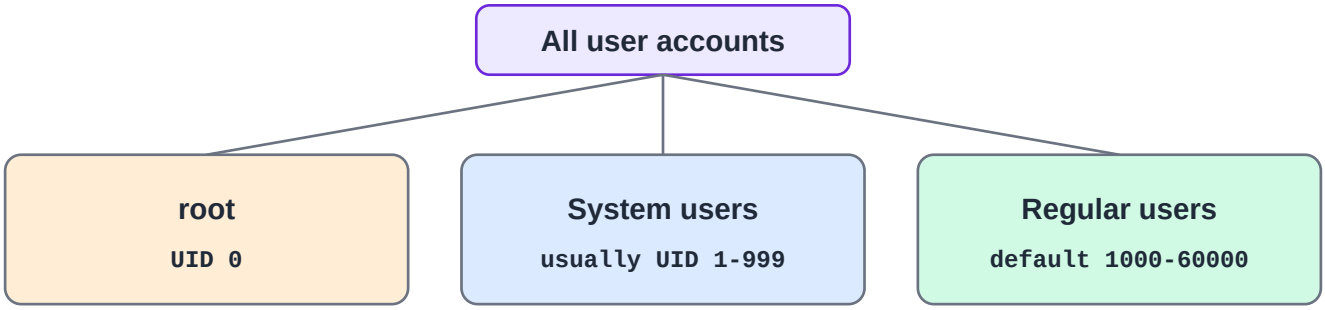
User management means creating, changing, locking and removing the accounts that can use a Linux system, and controlling what each one may do. Every person or service that touches the system runs as a **user**. Linux identifies a user by a number, the **UID**; the name is only a label for it.

The account data is kept in plain text files. Two matter most: **/etc/passwd** (who the users are) and **/etc/shadow** (their hashed passwords and password ageing).

Analogy: /etc/passwd is the school register everyone may read (name, roll number, class, locker). /etc/shadow is the principal's locked cabinet that holds each student's secret key and its expiry dates.

Types of users

Type	UID	Purpose	Login shell
root (superuser)	`0`	Full control of the system. Counted as a system user because the OS creates it, but it is the only one with unlimited power.	`/bin/bash`
System users	`usually 1 - 999`	Created by the OS or by packages to run services in the background (for example daemon, sshd, www-data, mysql). Nobody is meant to log in as them.	`often a nologin shell`
Regular (normal) users	`default 1000 - 60000`	Real people. Get a home directory, a shell and a password.	`/bin/bash`



UID ranges are configurable defaults. login.defs(5) documents the defaults UID_MIN 1000 and UID_MAX 60000 for regular users, and SYS_UID_MIN 101 with SYS_UID_MAX = UID_MIN-1 (999) for system users. They are set in /etc/login.defs, so a system can use different values. The ranges in this document are defaults and a common rule of thumb, not fixed rules. Check yours: `grep -E "^(SYS_)?UID_(MIN|MAX)" /etc/login.defs`

Check who you are

Identity commands - any user

```
whoami          # current username
id              # UID, primary GID and all groups
id <user>       # same for another user
who             # who is logged in now
last            # login history
```

2. /etc/passwd - the user database

Every account has **one line** in /etc/passwd. The file is readable by everyone, because programs such as ls need it to turn a UID into a name. Each line has **7 fields** separated by a colon.

Example line

```
alice:x:1001:1001:Alice Rao,Room 5:/home/alice:/bin/bash
```

alice	x	1001	1001	Alice Rao	/home/alice	/bin/bash
1	2	3	4	5	6	7
Username	Password placeholder	UID	Primary GID	Comment (GECOS)	Home directory	Login shell

#	Field	Meaning
1	Username	Login name, up to 32 characters. Lowercase letters, digits, underscore and hyphen are the safe choice.
2	Password	Normally **x**, meaning the real hash is in /etc/shadow. Historic systems stored the hash here.
3	UID	Numeric user ID. 0 is root. The kernel uses this number, not the name.
4	GID	Numeric ID of the **primary group**. Names for GIDs live in /etc/group.
5	GECOS / comment	Free text: full name, office, phone. Set with useradd -c or usermod -c.
6	Home directory	Where the user lands after login, for example /home/alice. Root uses /root.
7	Login shell	Program started at login. /usr/sbin/nologin (or /sbin/nologin) refuses interactive login.

Reading specific lines with head and tail

/etc/passwd can be long. Use head to see the first lines, tail to see the last lines, and combine them to pick one line. New accounts are appended at the bottom, so tail shows the newest users.

head / tail / cut - any user

```
head -n 5 /etc/passwd                # first 5 lines
tail -n 3 /etc/passwd                # last 3 lines (newest users)
head -n 10 /etc/passwd | tail -n 1   # exactly line 10
tail -f <logfile>                    # follow a file live (Ctrl+C to stop)
cut -d: -f1 /etc/passwd              # list only usernames (field 1)
cut -d: -f1,3 /etc/passwd            # username and UID
getent passwd alice                  # look up one user
```

Common slip: the option for a line count is **-n** (head -n 5), not -s.

3. /etc/shadow - passwords and ageing

/etc/shadow stores the **hashed** password and the password-ageing rules. It exists so ordinary users can read /etc/passwd without being able to copy password hashes. Per passwd(5), it is readable by the superuser only. Each line has **9 fields**.

Example line

```
alice:$6$Xk3...hash...:20751:0:90:7:14:20900:
```

alice	6...	20751	0	90	7	14	20900	(empty)
1	2	3	4	5	6	7	8	9
Username	Hashed password	Last change	Min days	Max days	Warn days	Inactive days	Expire date	Reserved

#	Field	Meaning
1	Username	Must match the name in /etc/passwd.
2	Password hash	Starts with the algorithm id (crypt(5)): 6 = sha512crypt, 5 = sha256crypt, y = yescrypt, 1 = md5crypt (not for new hashes). A leading **!** means the password is locked (the rest is the old hash). A value that is not a valid hash, such as **!** or *****, means no UNIX password can be used to log in. Other login methods, such as SSH keys, may still work.
3	Last change	Days since 1 January 1970 when the password was last changed. 0 forces a change at next login.
4	Minimum age	Days that must pass before the user may change the password again.
5	Maximum age	Days the password stays valid. After that it must be changed.
6	Warning period	Days before expiry that the user is warned at login.
7	Inactivity period	Days after expiry during which the user can still log in and change the password. After that the account is disabled.
8	Account expiry	Day number (since 1970) when the whole account is disabled.
9	Reserved	Empty. Kept for future use.

Remember: /etc/passwd has **7** fields and /etc/shadow has **9**. The two files are linked by the username in field 1.

	/etc/passwd	/etc/shadow
Holds	Account details	Password hash and ageing
Fields	7	9
Access (passwd(5))	Read permission for all users, write only for the superuser	Readable by the superuser only
Who reads it	Everyone, every login tool	Root and the passwd and login programs
Group equivalent	/etc/group	/etc/gshadow

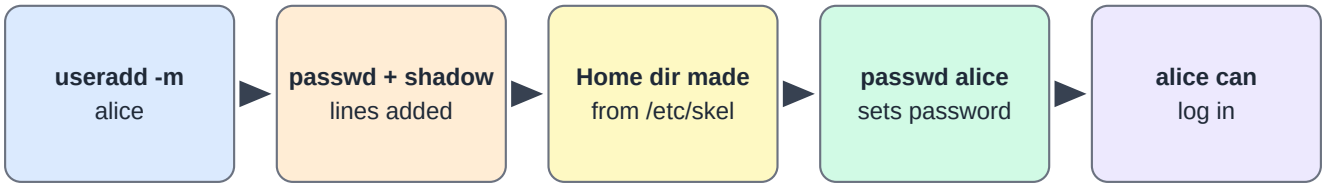
Inspect the files - cat /etc/shadow needs ROOT

```
ls -l /etc/passwd /etc/shadow
cat /etc/passwd
cat /etc/shadow
```

4. Creating a user - useradd

useradd adds the line to /etc/passwd and /etc/shadow, creates a same-name personal group when USERGROUPS_ENAB is yes in /etc/login.defs (or when -U is given), and creates the home directory with -m or when CREATE_HOME is set. It does **not** set a usable password, so the account stays locked until you run passwd.

What happens when you create a user



useradd - ROOT

```
useradd -m -U alice                    # home + same-name group
useradd -m -s /bin/bash alice          # also choose the login shell
useradd -m -c "Alice Rao" alice        # add a comment (full name)
useradd -m -u 1500 -g developers alice # fixed UID and primary group
useradd -m -G sudo,docker alice        # add supplementary groups
useradd -r -s /usr/sbin/nologin svc    # system account for a service
useradd -m -e 2026-12-31 intern        # account expires on a date
useradd -D                             # show default settings
```

Option	Meaning
`-m`	Create the home directory and copy files from /etc/skel into it. Without -m, the home is created only if CREATE_HOME is set in login.defs.
`-U / -N`	-U creates a same-name group for the user. -N does not. If neither is given, USERGROUPS_ENAB in login.defs decides.
`-d`	Use this directory as home instead of /home/name.
`-s`	Login shell, for example /bin/bash.
`-u`	Set the UID explicitly.
`-g`	Primary group (name or GID).
`-G`	Supplementary groups, comma separated with no spaces.
`-c "text"`	Comment field (full name).
`-e`	Account expiry date, YYYY-MM-DD.
`-r`	Create a system account (UID from the system range, normally no home).

Debian: adduser is a separate, friendlier command. Per the Debian adduser(8) page it copies /etc/skel into the home directory and prompts for the comment and a password. useradd is the low-level shadow-utils tool.

5. Passwords - passwd

passwd sets or changes a password and writes the hash into field 2 of /etc/shadow. A normal user may change only their own password. Root can change anyone's without knowing the old one.

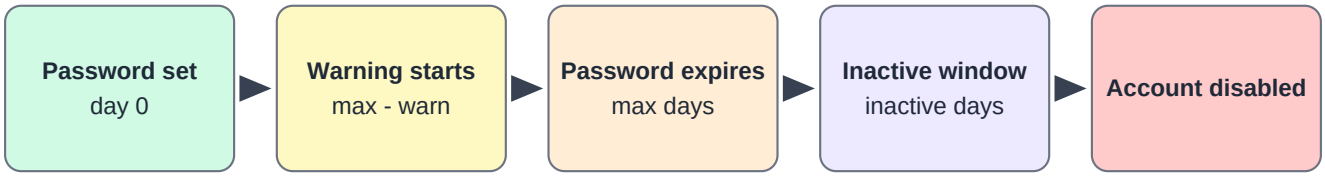
passwd

```
passwd              # change your own password  (ANY USER)
passwd alice        # set or change alice's password  (ROOT)
passwd -S alice     # status: locked / has password / none
passwd -l alice     # lock the password (puts ! in front of the hash)
passwd -u alice     # unlock it
passwd -e alice     # expire now: must change at next login
passwd -d alice     # delete the password (login with none - avoid)
passwd -n 1 -x 90 -w 7 -i 14 alice   # min, max, warn, inactive days
```

Locked password is not a disabled account. passwd(1) says passwd -l does not disable the account: the user may still log in with another authentication token, such as an SSH key. The documented way to disable the account is an account expiry date in the past: **`usermod -e 1 alice`** (sets expiry to 1970-01-02). Account expiry is checked through the login stack (PAM), so test it on your own system for services such as SSH.

Password ageing - chage

chage (change age) edits the ageing fields of /etc/shadow, fields 3 to 8. Anyone may run chage -l on their own account; changing values needs root.



chage - ROOT

```
chage -l alice           # list ageing info (own account: ANY USER)
chage -M 90 alice         # max days between changes
chage -m 7 alice          # min days between changes
chage -W 14 alice         # warn 14 days before expiry
chage -I 30 alice         # days of grace after expiry
chage -E 2026-12-31 alice # account expiry date
chage -d 0 alice          # force password change at next login
```

chage option	-d	-m	-M	-W	-I	-E
Shadow field	3 Last change	4 Min age	5 Max age	6 Warn days	7 Inactive	8 Expiry

6. Modifying and deleting users

Modify - usermod

usermod edits an existing account. Do not change a user who is logged in without warning them.

usermod - ROOT

```
usermod -c "Alice R" alice        # change the comment
usermod -s /bin/zsh alice          # change the login shell
usermod -l alicia alice           # rename the login (home dir not renamed)
usermod -d /home/alicia -m alicia # move the home directory
usermod -aG developers alice      # ADD to a group, keep the others
usermod -g staff alice            # change the primary group
usermod -L alice                  # lock the password (! before hash)
usermod -U alice                  # unlock it
usermod -e 2026-12-31 alice       # set account expiry date
```

-aG versus -G: usermod -G developers alice **replaces** all her supplementary groups with just developers. Always use **-a** with **-G** to append.

Delete - userdel

userdel - ROOT

```
userdel bob             # remove the account, keep the home directory
userdel -r bob          # remove account, home directory and mail spool
userdel -f bob          # force, even if logged in (use with care)
deluser bob             # Debian helper command
```

> Check for running processes and files owned by the user first. If you keep the files, they stay owned by the old UID, and a future user given that UID would inherit them. List them with: **`find / -user bob 2>/dev/null`**

Home directories

A regular user's home is normally **/home/<name>**, owned by that user. When you use useradd -m, the files in **/etc/skel** (such as .bashrc and .profile) are copied into it, so every new user starts with the same defaults. Root's home is /root.

Home directory commands

```
ls -ld /home/alice          # owner and permissions of a home
ls -la /etc/skel             # template files for new homes
mkdir /etc/skel/Documents    # every NEW user gets this folder  (ROOT)
chown -R alice:alice /home/alice   # repair ownership  (ROOT)
chmod 750 /home/alice         # others cannot read the home  (ROOT/owner)
```

7. Checking users and safe editing

Query commands

Look up users - any user

```
getent passwd alice            # one user from the account database
getent passwd | cut -d: -f1    # all usernames
awk -F: '$3>=1000 && $3<=60000 {print $1}' /etc/passwd  # default range
grep nologin /etc/passwd         # service accounts that cannot log in
grep -c "" /etc/passwd           # number of accounts
id alice                         # UID, GID, groups
groups alice                     # group names
last -n 5                        # last 5 logins
lastlog                          # last login of every user
```

Safe editing

Safe editing - ROOT

```
vipw            # safely edit /etc/passwd (locks the file)
vipw -s         # safely edit /etc/shadow
pwck            # check passwd and shadow for errors
grpck           # check group files
```

Never open /etc/passwd or /etc/shadow in a normal editor on a live system. vipw locks the file so two edits cannot collide, and pwck catches broken lines before they lock people out.

Key points to remember

- Three kinds of users: root (UID 0), system users (usually 1-999) and regular users (default 1000-60000). The ranges are set in /etc/login.defs.
- /etc/passwd = 7 fields, readable by all. /etc/shadow = 9 fields, root only.
- An x in passwd field 2 means the hash lives in shadow. A leading ! in shadow means locked.
- useradd -m creates the home from /etc/skel. The account stays locked until passwd sets a password.
- usermod needs -a with -G, or it replaces the supplementary groups.
- userdel -r removes the home directory too. Plain userdel keeps it.
- chage -d 0 forces a password change at next login. chage -l shows ageing.
- head -n and tail -n select lines. Edit account files with vipw, then run pwck.

8. Command chart

Command	What it does	Needs
`useradd -m name`	Create a user with a home directory	**ROOT**
`useradd -r name`	Create a system user	**ROOT**
`passwd name`	Set or change a password	**ROOT**, own: **ANY USER**
`passwd -S name`	Show password status	**ROOT**
`passwd -l / -u name`	Lock / unlock the password	**ROOT**
`passwd -e name`	Force change at next login	**ROOT**
`chage -l name`	Show ageing details	**ROOT**, own: **ANY USER**
`chage -M / -m / -W / -I / -E`	Max, min, warn, inactive, expiry	**ROOT**
`chage -d 0 name`	Force change at next login	**ROOT**
`usermod -aG grp name`	Append to a supplementary group	**ROOT**
`usermod -s / -c / -l / -d`	Change shell, comment, login, home	**ROOT**
`usermod -L / -U name`	Lock / unlock the password	**ROOT**
`userdel name`	Delete account, keep home	**ROOT**
`userdel -r name`	Delete account and home	**ROOT**
`id / whoami / groups`	Show identity and groups	**ANY USER**
`who / w / last`	Who is or was logged in	**ANY USER**
`head -n N file`	First N lines	**ANY USER**
`tail -n N file`	Last N lines	**ANY USER**
`tail -f file`	Follow a file live	**ANY USER**
`cut -d: -f1 /etc/passwd`	List usernames	**ANY USER**
`getent passwd name`	Look up a user	**ANY USER**
`cat /etc/passwd`	View account file	**ANY USER**
`cat /etc/shadow`	View password file	**ROOT**
`vipw / pwck`	Safe edit / check account files	**ROOT**

9. Interview questions with model answers

Q1. What is the difference between system and regular users?

System users are created by the OS or packages to run services; their UIDs are usually below 1000 and they often have a nologin shell. Regular users are real people, with UIDs from UID_MIN (default 1000) up to UID_MAX, a home and a shell. The ranges are set in /etc/login.defs.

Q2. Is root a system user? What is its UID?

Root is the superuser with UID 0. It is created by the OS so it is counted among system users, but it has unlimited privileges.

Q3. What does /etc/passwd contain and how many fields?

One line per account with 7 fields: username, password placeholder, UID, GID, comment, home directory, shell.

Q4. Why is there an x in the password field?

It means the real hash is stored in /etc/shadow, which only root can read.

Q5. What is /etc/shadow and how many fields does it have?

It holds the password hashes and ageing data, in 9 fields: username, hash, last change, min, max, warn, inactive, expire, reserved.

Q6. Why were passwords moved out of /etc/passwd?

/etc/passwd must be world-readable. Keeping hashes there let anyone copy them and try to crack them offline.

Q7. What does ! or * in the hash field mean?

A leading ! means the password is locked. A value like ! or * is not a valid hash, so no UNIX password works. Other login methods may still be possible.

Q8. How do you list regular users?

There is no single flag. Read UID_MIN and UID_MAX in /etc/login.defs, then filter by that range, for example awk -F: '$3>=1000 && $3<=60000 {print $1}' /etc/passwd. A plain >= 1000 is only a rough filter: it also lists accounts above UID_MAX, such as nobody (65534 on many systems).

Q9. Difference between useradd and adduser?

useradd is the low-level shadow-utils command. adduser is a Debian command that copies /etc/skel and prompts for the comment and a password.

Q10. Why add -m to useradd?

It creates the home directory and copies /etc/skel into it. Without -m the home is created only if CREATE_HOME is set in /etc/login.defs.

Q11. What is /etc/skel?

The template directory whose files are copied into every new home directory.

Q12. How do you set a password for a new user?

passwd username, run as root. useradd alone leaves the account locked.

Q13. Difference between passwd -l and usermod -L?

Both put ! before the hash and disable only password login. passwd(1) says to disable the account use usermod -e 1 (expiry in the past). Check that it works for your services, such as SSH.

Q14. What does chage do and how do you force a password change at first login?

chage manages password ageing. chage -d 0 username forces a change at the next login.

Q15. How do you add a user to a group without removing others?

usermod -aG groupname username. Without -a, -G replaces the supplementary groups.

Q16. Difference between userdel and userdel -r?

userdel removes the account only. -r also deletes the home directory and mail spool.

Q17. How do you view the first and last lines of a file?

head -n 5 file and tail -n 5 file. tail -f follows a growing file, such as a log.

Q18. How do you show line 10 of /etc/passwd?

head -n 10 /etc/passwd | tail -n 1, or sed -n 10p /etc/passwd.

Q19. How do you edit /etc/passwd safely?

Use vipw (and vipw -s for shadow); it locks the file. Validate afterwards with pwck.

Sources (checked 26 Oct 2026)

shadow(5): fields, ! and * meaning, account vs password expiry
https://man7.org/linux/man-pages/man5/shadow.5.html

passwd(5): 7 fields
https://man7.org/linux/man-pages/man5/passwd.5.html

login.defs(5): UID_MIN, UID_MAX, SYS_UID_MIN, SYS_UID_MAX
https://man7.org/linux/man-pages/man5/login.defs.5.html

useradd(8)
https://man7.org/linux/man-pages/man8/useradd.8.html

usermod(8)
https://man7.org/linux/man-pages/man8/usermod.8.html

userdel(8)
https://man7.org/linux/man-pages/man8/userdel.8.html

passwd(1): -l does not disable the account
https://man7.org/linux/man-pages/man1/passwd.1.html

chage(1)
https://man7.org/linux/man-pages/man1/chage.1.html

vipw(8)
https://man7.org/linux/man-pages/man8/vipw.8.html

pwck(8)
https://man7.org/linux/man-pages/man8/pwck.8.html

crypt(5): hash prefixes 6 5 y 1
https://manpages.debian.org/bookworm/libcrypt-dev/crypt.5.en.html

adduser(8), Debian
https://manpages.debian.org/bookworm/adduser/adduser.8.en.html